DAA LABWORK WEEK 5

BY

A.JEEVITESWARA REDDY

CH.SC.U4CSE24164

CODE:

QUICK SORT Taking first element as pivot

```
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partitionFirst(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low + 1;
    int j = high;

    while (i <= j) {
        while (i <= high && arr[i] <= pivot) i++;
        while (j >= low && arr[j] > pivot) j--;

        if (i < j)
            swap(&arr[i], &arr[j]);
    }
    swap(&arr[low], &arr[j]);
    return j;
}

void quickSortFirst(int arr[], int low, int high) {
    if (low < high) {
        int pi = partitionFirst(arr, low, high);
        quickSortFirst(arr, low, pi - 1);
        quickSortFirst(arr, pi + 1, high);
    }
}

void printArray(int arr[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original:\n");
    printArray(arr, n);
```

```
int main() {
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original:\n");
    printArray(arr, n);

    quickSortFirst(arr, 0, n-1);

    printf("Sorted (First Pivot):\n");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
C:\Users\A JEEVITESWARA REDDY\OneDrive\Desktop>qs1
Original:
157 110 147 122 111 149 151 141 123 112 117 133
Sorted (First Pivot):
110 111 112 117 122 123 133 141 147 149 151 157
```

ALGORITHM:

1. Pick the first number as the pivot.
2. Put all smaller numbers on the left side of the pivot.
3. Put all bigger numbers on the right side.
4. Now the pivot is in its correct sorted place.
5. Repeat the same steps for the left part and right part.

CODE:

QUICK SORT Taking last element as pivot

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partitionLast(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i+1], &arr[high]);
    return i+1;
}
```

```c
int main() {
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original:\n");
    printArray(arr, n);

    quickSortLast(arr, 0, n-1);

    printf("Sorted (Last Pivot):\n");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
C:\Users\A JEEVITESWARA REDDY\OneDrive\Desktop>qs1
Original:
157 110 147 122 111 149 151 141 123 112 117 133
Sorted (Last Pivot):
110 111 112 117 122 123 133 141 147 149 151 157
```

ALGORITHM:

1. Pick the last number as the pivot.
2. Compare every element with the pivot.
3. Move all smaller or equal values to the left side.
4. Move all greater values to the right side.
5. Do the same process again for the two divided parts.

CODE:
QUICK SORT Taking random element as pivot

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partitionLast(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i+1], &arr[high]);
    return i+1;
}

int partitionRandom(int arr[], int low, int high) {
    int randomIndex = low + rand() % (high - low + 1);
    swap(&arr[randomIndex], &arr[high]);
    return partitionLast(arr, low, high);
}

void quickSortRandom(int arr[], int low, int high) {
    if(low < high) {
        int pi = partitionRandom(arr, low, high);
        quickSortRandom(arr, low, pi-1);
        quickSortRandom(arr, pi+1, high);
    }
}

void printArray(int arr[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```c
int main() {
    srand(time(NULL));

    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original:\n");
    printArray(arr, n);

    quickSortRandom(arr, 0, n-1);

    printf("Sorted (Random Pivot):\n");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
C:\Users\A JEEVITESWARA REDDY\OneDrive\Desktop>qs1
Original:
157 110 147 122 111 149 151 141 123 112 117 133
Sorted (Random Pivot):
110 111 112 117 122 123 133 141 147 149 151 157
```

ALGORITHM:

1. Choose a random index between low and high using
   randomIndex = low + rand() % (high - low + 1)
2. Take the element at that random index as the pivot.
3. Rearrange the array so all elements smaller than pivot go to the left side.
4. Rearrange the array so all elements greater than pivot go to the right side.
5. Repeat the same steps for the left and right subarrays until the whole array is sorted.

REASONING:

First element pivot is simple, but if the array is already sorted (or nearly sorted), it gives very unbalanced partitions and becomes slow (worst case $O(n^2)$).

Last element pivot has the same problem as first pivot, it can also become worst case $O(n^2)$ on sorted or reverse sorted data.

Random pivot usually gives more balanced partitions because the pivot is unpredictable, so it avoids bad cases most of the time.

Balanced partitions make Quick Sort run close to its best/average time $O(n \log n)$.

So in real practice, random pivot is considered the most optimal among these three

WORKING:

FOR taking first element as pivot :

Quick sort :-

1) First Element as pivot

↓Pivot

157 110 147 122 111 149 151 141 123 112 117 133

Step 1:-

↓Pivot

133 110 147 122 111 149 151 141 123 112 117 157

Step 2:- Replace 147 and 117

133 110 117 122 111 149 151 141 123 112 147 157

↑ Pivot

Step 3:- Replace 149 and 112

133 110 117 122 111 112 151 141 123 149 147 157

↑ Pivot

Step 4:- Replace 151 and 123

133 110 117 122 111 112 123 141 151 149 147 157

↑ Pivot

Step 5:- Replace 133 and 123

↗ sorted

123 110 117 122 111 112 [133] [141 151 149 147 157

↑ Pivot

Step 6:- Replace 123 and 112

112 110 117 122 111 123 133 141 151 149 147 157

↑ Pivot

Step 7:- Replace 117 and 111

↑ Pivot

Replace 147 and 151

112 110 111 122 117 123 133 141 147 149 151 157

↑ Pivot

Step 8:- Replace 112 and 111

↑ Pivot

111 110 112 122 117 123 133 141 147 149 151 157

↓ Pivot

swap

Step 9:- 110 111 112 117 122 123 133 141 147 149 151 157

FOR taking last element as pivot:

It takes 9 steps to comp
first element as pivot

ii) Last Element as Pivot :-

Step 1:- 157 110 147 122 111 149 151 141 123 112 117 133 (Pivot)

Replace 157 and 117

Step 2:- 117 110 147 122 111 149 151 141 123 112 157 133 (Pivot)

swap

Step 3:- 117 110 112 122 111 149 151 141 123 147 157 133 (Pivot)

swap

Step 4:- 117 110 112 122 111 123 151 141 149 147 157 133 (Pivot)

swap

Step 5:- 117 110 112 122 111 123 133 141 149 147 157 151

Pivot, sorted, Pivot, swap

Step 6:- 117 110 112 122 111 123 133 141 149 147 151 157

Pivot, sorted, Pivot, sorted, swap

Step 7:- 110 117 112 122 111 123 133 141 147 149 151 157

Pivot, sorted, swap

Step 8:- 110 111 112 122 117 123 133 141 147 149 151 157

Pivot, Pivot, swap

Step 9:- 110 111 112 117 122 123 133 141 147 149 151 157

Pivot, Pivot

Step 10:- 110 111 112 117 122 123 133 141 147 149 151 157

It takes 10 steps to sort the unsorted

FOR taking random element as pivot:

(iii) Random element

Pivot = 149

[110, 149, 122, 111, 141, 123, 112, 117, 133, [149], 157, 151]

Pivot = 122

[110, 111, 112, 117, [122], 149, 141, 123, 133, 149, 157, 151]

Pivot = 133

[110, 111, 123, 117, 122, 123, [133], 149, 141, 149, 157, 151]

Pivot = 141

[110, 111, 112, 119, 122, 123, 133, [141], 147, 149, 157, 151]

Pivot = 151

[110, 111, 112, 117, 122, 123, 133, 141, 149, 149, [151], 157]

Pivot = 157

[110, 111, 112, 117, 122, 123, 133, 141, 147, 149, 151, 157]

It is sorted in 6 steps.